Postmortem, Exam Review

CSE 493L: Concurrency, Parallelism, and Rust

outline

cheat sheet handwritten

registration filled up in 1 minute?

how to use knowledge in other languages (rust to c)

what is research, my plug for grad school

practice problems

understand my code examples

meta discussion

Administrivia

HW9 late due date today

Survey deadline today (1% bonus point!)

Final Exam Monday

RESPONSES
(41/48) 85% very high

Agenda

Exam Info

Rust to C

Research plug

Practice problems

Meta discussion (10 mins?)

Final Exam Info

Monday (3 days away) @ 8:30AM, Gates G10 (this room!)

Arrive at 8:20, we start on time

Bring:

- *Handwritten* cheat sheet
- Writing utensils and backups (I don’t have a pencil sharpener)
- Water, quiet snacks

Length: 110 minutes (sharp deadline)

- 32 multiple choice, 1pt each
- 3 FRQ, 4pts each
- Total: 44pts

Final Exam Study Tips

- Do the practice exam with a stopwatch
- Go over quizzes
- Look through lecture slides for code examples
  - Focus on what we spent a lot of time on
  - Examples: coroutines, interleavings, data races, synch primitives implementation, …
  - Understand why examples were correct or wrong (e.g. good mutex, bad mutex. Rust Linked List / recursive data structures)

Final Exam Test Taking Tips

- Some multiple choice questions are much, much harder than others
- If you are worried about time, skip the hard questions until last!
  - 32 multiple choice * 2 minutes per => 64 minutes total
  - 3 FRQ * 8 minutes per => 24 minutes total
  - 20 minute buffer to check through questions

Don’t freak out! A hard exam will be curved! And ask questions if you’re stuck!

What did we learn?

- Concurrent abstractions
  - Coroutines, threads, synchronization, thread pools, async/await, MapReduce/Spark
  - Deadlock, Model checking
- Parallelism / Architecture
  - Multicore, core specialization, cache coherence, memory consistency, GPUs, pipeline parallel
- Rust
  - Foundations (ownership, borrowing, lifetimes, safety & soundness)
  - Custom types (structs, enums, traits, generics)
  - Standard library types (Box, Vec, Arc, Mutex)
  - Fearless concurrency (Send, Sync, different types of closures, async/await)

How do I get better at Rust?

Don’t stop using it!

Your next side project? Give Rust a try!

Final project in another class? Do it in Rust!

Leetcode? Rust!

Wherever you go…

You can use Rust principles!

The ideas can still give you memory safety, even if the compiler isn’t there

```
int address;
```

```
struct Address {
    int inner;
}
```

Code C like Rust

```
int add_2(int * num);
```

```
void main() {
    int * my_num = malloc() and set to 5;
    add_2(my_num);
    print(my_num); }
```

Code C like Rust

```
// takes ownership of num
int add_2(int * num) {
    int ret = *num + 2;
    free(num);
    return ret; }
void main() {
    int * my_num = malloc() and set to 5;
    add_2(my_num);
    print(my_num); }
```

Code C like Rust

```
// borrows num immutably
int add_2(const int * num) {
    int ret = *num + 2;


    return ret; }
void main() {
    int * my_num = malloc() and set to 5;
    add_2(my_num);
    print(my_num); }
```

Code C like Rust

```
// borrows num mutably
int add_2(int * num) {
    *num += 2;


    return *num; }
void main() {
    int * my_num = malloc() and set to 5;
    add_2(my_num);
    print(my_num); }
```

Code C like Rust

```
// borrows num mutably
int add_2(int * num_mut) {
    *num_mut += 2;


    return *num_mut; }
void main() {
    int * my_num_owned = malloc() and set to 5;
    add_2(my_num_owned);
    print(my_num_owned); free(my_num_owned); }
```

Code C like Rust

Think about ownership!

Think about lifetimes!

How did I learn Rust?

In Jan 2024, I knew 0 Rust. Now, I'm teaching this class. How?

I joined a project writing an OS kernel using Rust

I worked with others who knew Rust

I did a lot of Googling

Isn't `Fn` just `FnOnce + Copy`?

help

KSXGitHub

play.rust-lang.org

Rust Playground

A browser interface to the Rust compiler to experiment with the language

```
pub fn call_twice<X, Y, F>(f: F, x1: X, x2: X) -> (Y, Y)
where
    F: FnOnce(X) -> Y + Copy,
{
    (f(x1), f(x2))
}
```

A quick plug: the value of a PhD

What is “research”?

- solving interesting problems
- identify the current bottleneck and redesign the compute stack to fix it
- identify the *next* bottleneck and redesign to fix that
- fix abstractions that you don’t like

What is my research?

I like to work at the hardware-software boundary layer.

What interfaces *could* hardware expose that would allow us to improve our software?

- Pika (Rust OS ft. containerization as a first class citizen)
- NEMO (a programmable memory controller enabling tiered memory)
- Google Workload Trace Analysis (likely will turn into cache design stuff)

If you’re interested, reach out!

Practice Problems

Questions from practice final

Kahoot:
https://create.kahoot.it/share/cse-493l-review/96e3f39a-d801-4e58-9d7b-45889b90adce

Meta Discussion

- Rust vs Concurrency vs Parallelism
- Lectures pacing
- Participation

Picture

& please take candy

Thank you

It has truly been my pleasure to get to create and teach this course from nothing

Shoutout to all of you, your participation has motivated me

Special thanks to my TAs and Tom Anderson